

STUDY GUIDE

# 주차 배정 알고리즘 공부 로드맵

벡터 하나만 알고 시작해서, 교수님과 대화가 되는  
지점까지.

무엇을 어떤 순서로 공부해야 하는지, 그리고 우리 주차장이  
수학적으로 정확히 무엇인지.

---

프로젝트 빔 프로젝터 주차 유도 시스템 개념검증

대상 도면 주차면 120면 · 통로 노드 71개 · 방향성 간선 80개 ·  $130 \times 97.5$  m

함께 보는 파일 `parking_topology.py` (단독 실행, 의존성 없음)

작성 2026-09-09

## 차례

### 1. 이 문서의 사용법

지금 아는 것에서 출발하는 법. 읽는 순서.

---

### 2. 큰 그림 — 주차 배정은 무슨 문제인가

한 문장의 질문이 네 개의 수학 분야에 걸쳐 있다.

---

### 3. 선수 지식 점검

벡터를 아는 상태에서 더 필요한 다섯 가지.

---

### 4. 분야별 공부 가이드

그래프 · 조합최적화 · 온라인 · 확률/큐잉 · 교통류 · MDP

---

### 5. 우리 주차장의 수학적 표현

실제 숫자와 수식. 붙여 넣고 돌릴 수 있는 코드.

---

### 6. 교수님께 가져갈 질문지

무엇을 보여드리고 무엇을 여쭙 것인가.

---

### 7. 4주 학습 계획과 자료 목록

무료 자료 우선.

---

# 1 이 문서의 사용법

모르는 것이 많은 게 문제가 아니라, 무엇을 모르는지 모르는 게 문제입니다. 이 문서는 그 지도를 그립니다.

## 이 문서가 가정하는 출발점

벡터의 덧셈과 크기, 내적 정도를 안다고 가정합니다. 그거면 충분합니다. 아래에서 다루는 모든 분야는 결국 “점들 사이의 거리를 재고, 그 거리를 합쳐서 최소로 만드는 방법”이며, 벡터는 그 첫 걸음이기 때문입니다.

### 먼저 알아둘 것

이 분야를 전부 마스터할 필요는 **없습니다**. 목표는 “교수님이 하시는 말씀을 알아듣고, 우리 문제가 어느 칸에 들어가는지 짚고, 다음에 무엇을 읽어야 할지 스스로 정할 수 있는 상태”입니다. 각 분야마다 **최소한 이것만** 절을 두었으니 그것부터 보세요.

## 읽는 순서

| 상황     | 이렇게 읽으세요                                    |
|--------|---------------------------------------------|
| 시간이 하루 | 2장(큰 그림) → 4장의 한 문장 요약만 훑기 → 6장(질문지)        |
| 일주일    | 2 → 3 → 4.1(그래프) → 4.2(배정) → 5장 코드 돌려보기 → 6 |
| 한 달    | 7장의 계획표대로                                   |

## 같이 있는 파일

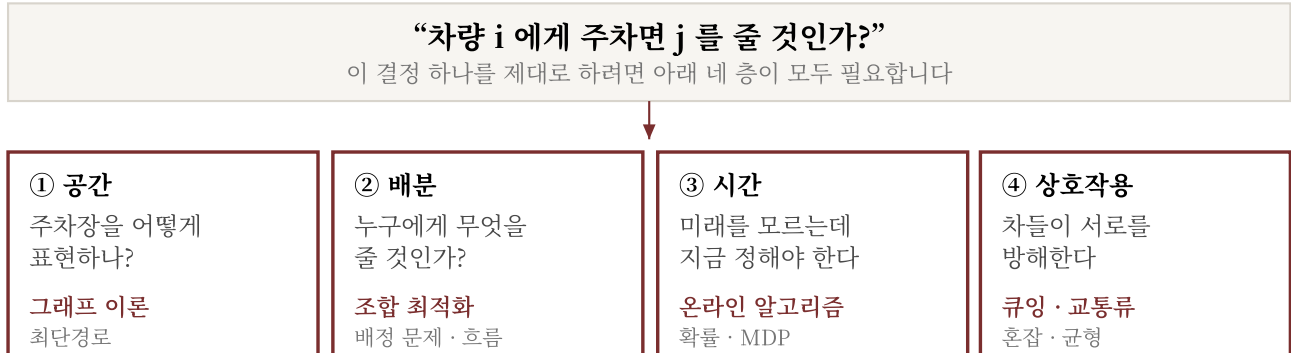
`parking_topology.py` 는 이 파일 하나만으로 돌아갑니다. 외부 라이브러리도, 나머지 프로젝트도 필요 없습니다. 우리 주차장의 실제 도면 데이터가 안에 들어 있고, 그래프 표현 · 최단경로 · 비용 행렬 · 배정 알고리즘 두 가지 · 혼잡 측정을 차례로 보여줍니다.

```
python parking_topology.py
```

교수님께 보여드릴 때 이 파일을 함께 열어 두시면 대화가 훨씬 구체적이 됩니다. “저희 주차장이 이렇게 생겼고, 지금 이렇게 풀고 있습니다” 를 한 화면에 보여드릴 수 있습니다.

## 2 큰 그림 — 주차 배정은 무슨 문제인가

“누구에게 어느 자리를 줄 것인가.” 이 한 문장이 네 개의 서로 다른 수학 분야에 걸쳐 있습니다.



### 지금 우리 프로젝트가 서 있는 자리

①②는 구현했고 ③은 걸쳐 있으며 ④는 거의 손대지 못했습니다. 그런데 “혼잡이 줄어드는가”는 ④의 질문입니다.

네 층은 따로 배울 수 있지만 따로 쓸 수는 없습니다. 배정(②)을 바꾸면 경로가 바뀌고, 경로가 바뀌면 통로 부하(④)가 바뀝니다. 배정 알고리즘은 사실 교통 배분 알고리즘이기도 합니다.

주차 배정 문제의 네 층. 아래로 갈수록 우리가 덜 다뤄 본 영역입니다.

## 왜 “가까운 자리를 주면 끝” 이 아닌가

직관적으로는 빈 자리 중 제일 가까운 곳을 주면 될 것 같습니다. 실제로 우리의 첫 알고리즘이 그것이었습니다. 그런데 곧 세 가지 문제가 드러납니다.

- “가깝다”가 두 개입니다. 차로 가까운 자리와 건물에서 가까운 자리가 다릅니다. 우리 도면에서 실제로 그렇습니다 — 주행 최단은 F-01, 비용 최소는 A-10 으로 갑니다.
- 지금 최선이 나중에 손해입니다. 좋은 자리를 일찍 소진하면 나중에 온 차량이 멀리 가야 하고, 그 이동이 통로 혼잡이 됩니다.
- 모두에게 좋은 자리를 주면 그 길이 막힙니다. 개인 최적의 합이 전체 최적이지 않습니다. 이게 4장 마지막에 나오는 무정부의 대가입니다.

### 3 선수 지식 점검

벡터를 아는 상태에서, 4장을 읽으려면 다섯 가지가 더 필요합니다. 각각 하루면 충분합니다.

| 필요한 것   | 최소한 이 정도                                                          | 모르면 어디서 막히나                                |
|---------|-------------------------------------------------------------------|--------------------------------------------|
| 집합과 표기  | $\in, \subseteq,  A , \Sigma, \operatorname{argmin}, \text{s.t.}$ | 최적화 문제 정의를 읽을 수 없습니다. 4.2의 수식이 전부 이 표기입니다. |
| 행렬      | 곱셈, 전치, 항등행렬. 벡터를 알면 반걸음입니다.                                      | 인접 행렬, 비용 행렬을 다룰 수 없습니다. 5장이 안 읽힙니다.       |
| 확률 기초   | 확률변수, 기댓값 $E[X]$ , 분산, 독립                                         | 도착이 무작위라는 사실을 모형에 넣을 수 없습니다. 4.4가 전부 막힙니다. |
| 점화식과 재귀 | $f(n) = f(n-1) + \dots$ 형태를 세우고 푸는 것                              | 동적계획법, 벨만 방정식(4.6)이 안 보입니다.                |
| 빅오 표기   | $O(n), O(n \log n), O(n^3)$ 의 뜻과 대소 비교                            | “이 알고리즘은 쓸 수 있나”를 판단할 수 없습니다.              |

#### 가장 먼저 할 일

**표기법부터 익히세요.** 수학이 어려운 게 아니라 표기가 낯설어서 못 읽는 경우가 대부분입니다. 아래 식을 소리 내어 읽을 수 있으면 3장은 통과입니다.

$$\begin{aligned} \min_x \quad & \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j \in J} x_{ij} = 1 \quad (\forall i \in I) \\ & \sum_{i \in I} x_{ij} \leq 1 \quad (\forall j \in J) \\ & x_{ij} \in \{0, 1\} \end{aligned}$$

첫 줄이 목적함수(최소로 만들 것), 아래 세 줄이 제약(반드시 지킬 것).  
기호 s.t. 는 *subject to*, “다음을 만족하면서”라는 뜻입니다.

“집합  $I$ 의 각 원소  $i$ 에 대해  $x_{ij}$ 를  $j$ 에 대해 모두 더하면 1이 되도록 하면서, 비용의 총합을 최소로 만드는  $x$ 를 찾아라.” — 우리말로 옮기면 “차량마다 자리를 정확히 하나씩 주되, 자리는 겹치지 않게, 총비용은 최소로.”

## 4 분야별 공부 가이드

여섯 분야. 각각 “왜 필요한가 → 최소한 이것만 → 우리 코드 어디에 → 교수님께 여쭙 것” 순서입니다.

### 4.1 그래프 이론과 최단경로

주차장을 “점과 화살표”로 바꾸면 거리 문제가 계산 가능한 문제가 된다.

왜 필요한가

주차장은 평면이지만 차는 아무 데나 갈 수 없습니다. 통로를 따라, 일방통행 방향으로만 갑니다. 이 제약을 담는 자료구조가 **방향 그래프**이고, “여기서 저기까지 얼마나 머냐”가 **최단경로 문제**입니다. 우리 도면은 노드 71개, 방향성 간선 80개입니다.

최소한 이것만

- $G = (V, E)$  표기, 방향/무방향, 가중치
- 인접 리스트 vs 인접 행렬 — 언제 무엇을 쓰나
- BFS · DFS (탐색), **다익스트라** (가중 최단경로,  $O(E \log V)$ )
- $A^*$  — 다익스트라에 “목적지 방향” 힌트를 준 것
- 강연결성 — 일방통행 때문에 “갈 수는 있는데 못 돌아오는” 구역이 생기나?

우리 코드 어디에

`sim/common/lotmap.py` 의 `successors()` , `shortest_path()` / `sim/control/routing.py` (회전 페널티가 붙은  $A^*$ ) / 동봉한 `parking_topology.py` 의 1~2절

교재

Kleinberg & Tardos, *Algorithm Design* 4장 (가장 읽기 쉽습니다) · CLRS *Introduction to Algorithms* 22~24장 (표준 참고서)

교수님께

1. 회전에 비용을 매기려면 그래프를 어떻게 바꿔야 합니까? (간선을 노드로 바꾸는 확장이 정석이라고 들었는데, 노드가 80개로 늘어도 괜찮습니까?)
2. 일방통행 격자에서 “강연결”을 보장하는 배치 조건이 알려져 있습니까?

### 4.2 조합 최적화 — 배정 문제

“누구에게 무엇을 줄까”의 정답을 다항시간에 찾을 수 있다. 단, 비용이 더하기로 표현될 때만.

왜 필요한가

이것이 **우리 문제의 심장**입니다. 차량 집합과 빈 자리 집합 사이의 **이분 그래프 매칭**이고, 비용을 최소화하는 매칭을 찾는 것이 **배정 문제(assignment problem)**입니다. 헝가리안 알고리즘이  $O(n^3)$  에

최적을 보장합니다.

최소한 이것만

- 이분 그래프, 매칭, 최대 매칭 / 최소비용 완전 매칭
- **헝가리안 알고리즘** — 원리는 나중에라도 “최적을 보장한다”는 사실은 지금
- 선형계획(LP)과 **쌍대성(duality)** — 왜 최적인지의 근거
- **완전 유니모듈러성** — 0/1 정수 문제인데 왜 실수로 풀어도 정수해가 나오나
- **최소비용 흐름(min-cost flow)** — 배정의 일반화. 간선에 용량을 줄 수 있다

우리 코드 어디에

`sim/control/cost.py` (비용 함수) · `sim/control/allocators/` (전략 4종) · `parking_topology.py` 의 3~4절 (헝가리안 구현 포함)

교재

Papadimitriou & Steiglitz, *Combinatorial Optimization* (Dover 판이 저렴) · Ahuja · Magnanti · Orlin, *Network Flows* (흐름 문제의 표준서)

교수님께

1. 배정 문제의 제약행렬이 완전 유니모듈러라 LP 완화로도 정수해가 나온다고 들었습니다. 저희가 혼잡 항을 **비선형**으로 바꾸면 그 성질이 깨질 텐데, 그때도 쓸 수 있는 방법이 있습니까?
2. 배정 문제를 최소비용 흐름으로 두고 통로 간선에 **용량**을 주면 혼잡을 자연스럽게 다룰 수 있을 것 같은데, 이 방향이 맞습니까?

## 4.3 온라인 알고리즘

*차는 한꺼번에 오지 않는다. 미래를 모르는 채 지금 정해야 하고, 그 손해를 수치로 잴 수 있다.*

왜 필요한가

4.2는 “모든 차량을 동시에 안다”고 가정합니다. 현실은 아닙니다. 차량은 하나씩 도착하고 배정은 되돌릴 수 없습니다. 이걸 다루는 분야가 **온라인 알고리즘**이고, “미래를 알았다면 얼마나 잘했을까”와 비교한 비율이 **경쟁비(competitive ratio)**입니다.

$$\text{competitive ratio} = \frac{\text{ALG}}{\text{OPT}}$$

ALG 는 하나씩 도착하는 대로 결정한 온라인 알고리즘의 비용, OPT 는 나중에 전부 알고 나서 푼 최적 비용입니다.

1 에 가까울수록 “미래를 몰라서 본 손해”가 적습니다.

우리 프로젝트에서 실제로 확인된 사실: 배치로 묶으려 했던 `hungarian_batch` 는 **배치가 한 번도 만들어지지 않아** 탐욕법과 같은 답을 냈습니다. 입구 대기가 뒷차 진입을 막기 때문입니다. **온라인**이라는

성질은 피할 수 있는 게 아니라 문제의 일부였습니다.

최소한 이것만

- 경쟁비의 정의, 최악 경우 분석
- 온라인 이분 매칭: 탐욕법은  $1/2$ , RANKING 알고리즘은  $1 - 1/e \approx 0.632$  (최적)
- 가중치가 붙으면 일반적으로 상수 경쟁비가 불가능하다는 사실

우리 코드 어디에

`sim/control/allocators/greedy_nearest.py` · `docs/DECISIONS.md` D-027

교재

Borodin & El-Yaniv, *Online Computation and Competitive Analysis* · Karp · Vazirani · Vazirani (1990) 원논문은 짧고 유명합니다

교수님께

1. 저희는 도착 분포를 알고 있습니다(시뮬레이터니까요). 분포를 아는 온라인 문제에서 최악 경우 경쟁비 말고 기대값 기준으로 성능을 보장하는 틀이 있습니까?

## 4.4 확률과정과 큐잉이론

주차장은 “서버가 120개인 대기 시스템”이다. 혼잡을 재려면 이 언어가 필요하다.

왜 필요한가

우리가 궁극적으로 묻는 것은 “혼잡이 줄어드는가”입니다. 혼잡은 개별 차량의 성질이 아니라 **계 전체의 성질**이고, 계를 다루는 언어가 큐잉이론입니다. 주차면 120개가 서버, 도착률이  $\lambda$ , 주차 후 머무는 시간이 서비스 시간입니다.

최소한 이것만

- 포아송 과정과 지수분포, 무기억성 — 왜 도착을 이걸로 모형화하나
- 리틀의 법칙  $L = \lambda W$  — 계 안의 평균 인원 = 도착률  $\times$  평균 체류시간. 증명이 거의 필요 없을 만큼 일반적이고, 그래서 강력합니다.
- M/M/c 대기 시스템, 그리고 대기실이 유한한 M/M/c/K
- 이용률  $\rho = \lambda/(c\mu)$  와 포화:  $\rho \rightarrow 1$  일 때 대기가 폭발한다

우리 데이터에 바로 적용해 보면

$$L = \lambda_{\text{eff}} \cdot W$$

L 계 안의 평균 대수 ·  $\lambda_{\text{eff}}$  실효 처리율(대/초) · W 평균 체류시간



| 주차 완료 | $\lambda_{\text{eff}}$ | $W$ (초) | $L$ (대) |
|-------|------------------------|---------|---------|
| 무안내   | 118.2                  | 0.131   | 53.7    |
| 안내    | 179.0                  | 0.199   | 28.4    |

busy 시나리오 · 도착률 0.20 대/초 · 900초 → 제공 부하 180대

통로에서 돌아다니는 차가 평균 7.05대에서 5.65대로 줄었습니다. 더 중요한 것은 **무안내**는 **180대 중 118대만 처리했다**는 사실입니다 — 이미 용량 밖입니다. 안내는 179대를 처리했으니 아직 여유가 있습니다.

교재

Sheldon Ross, *Introduction to Probability Models* (포아송·큐잉 표준서) · 한국어 자료로는 “확률과정” 또는 “대기행렬이론” 강의노트

교수님께

1. 주차장을 M/M/c/K 로 근사하면  $c$  와  $K$  는 각각 무엇으로 두어야 합니까? 주차면 수가  $c$  인 것 같은데, 통로를 도는 시간은 서비스인가요 대기인가요?
2. 이 근사가 **깨지는 지점**은 어디입니까? 저희는 차들이 서로를 물리적으로 막는데, 표준 큐잉 모형에는 그런 상호작용이 없습니다.

## 4.5 교통류 이론과 네트워크 균형

**이 시스템의 정체는 “안내판”이 아니라 사회적 최적을 실행 가능하게 만드는 장치다.**

왜 필요한가

혼잡을 정면으로 다루는 분야입니다. 그리고 여기서 **이 프로젝트의 가장 강한 논지**가 나옵니다 — 아래 균형 항목을 보세요.

최소한 이것만

- 기본 관계식  $q = k \cdot v$  (유량 = 밀도  $\times$  속도) 와 기본도: 밀도가 늘면 속도가 줄고, 어느 지점을 넘으면 유량마저 줄어든다 → 그 꼭짓점이 **용량**
- BPR 함수 — 혼잡을 비선형으로 값매기는 표준 방식

$$t_e(x) = t_e^0 \left[ 1 + \alpha \left( \frac{x}{u_e} \right)^\beta \right]$$

$x$  간선  $e$  의 부하 ·  $u_e$  그 간선의 용량 · 보통  $\alpha = 0.15, \beta = 4$

핵심은  $u_e$ , 즉 **용량**입니다. 부하가 용량에 가까워지면 비용이 폭발합니다. 지금 우리 비용 함수에는 용량 개념이 없습니다.

- Wardrop 균형 — 이 분야의 핵심 개념

|                |                                  |
|----------------|----------------------------------|
| 사용자 최적<br>(UE) | 각자 자기 비용만 최소화한다. = 우리의 무안내 베이스라인 |
|----------------|----------------------------------|

|                |                           |
|----------------|---------------------------|
| 사회적 최적<br>(SO) | 전체 비용의 합을 최소화한다. = 유도선 안내 |
|----------------|---------------------------|

둘의 비율이 무정부의 대가(Price of Anarchy)입니다.

- 브라에스의 역설 — 길을 더 놓았는데 전체가 느려지는 현상

이 틀이 왜 중요한가

주차장에서 사회적 최적을 아는 것은 쉽습니다. 관제는 모든 빈자리를 알고 있으니까요. 어려운 것은 그것을 운전자에게 실행시키는 것이고, 바닥에 그려진 유도선이 바로 그 실행 수단입니다.

그리고 이 틀이 저희 연구의 두 주제를 하나로 묶습니다: 비협조 운전자란 사회적 최적 배분에서 이탈해 사용자 최적으로 되돌아가는 사람이고, 곧 무정부의 대가를 다시 끌어올리는 요인입니다. 복귀 전략 비교는 “어느 전략이 PoA 를 가장 낮게 유지하는가”로 다시 쓸 수 있습니다.

교재

Yosef Sheffi, *Urban Transportation Networks* — 저자가 전문을 무료 공개. 이 분야의 표준서이고 Wardrop 균형·BPR·네트워크 배분이 전부 들어 있습니다.

교수님께

1. 도로망에서 쓰는 기본도가 주차장 통로에도 성립합니까? 속도가 훨씬 낮고, 주차하려고 멈추는 차가 흐름을 끊는데요.
2. 저희 시스템은 SO 를 강제할 수 있는 드문 경우라고 생각합니다. 이걸 정량화하려면 PoA 를 어떻게 측정해야 합니까? 분모(사후 최적)를 어떻게 계산합니까?

## 4.6 (선택) 마르코프 결정과정

“지금 이 자리를 주면 나중에 곤란해지는가”를 비용에 넣는 방법.

왜 필요한가

지금 비용 함수에는 자리의 기회비용이 없습니다. 그래서 좋은 자리를 이른 시각에 소진하고, 나중에 온 차량이 멀리 갑니다. 이 “미래 가치”를 다루는 틀이 MDP 이고, 항공권·호텔의 재고 관리와 같은 구조입니다.

$$\tilde{c}(i, j) = c(i, j) + \lambda_j$$

$\lambda_j$  는 자리  $j$  의 기회비용 — “지금 이 자리를 내주면 앞으로 얼마나 손해인가”

최소한 이것만

- 상태 · 행동 · 보상 · 정책, 할인율  $\gamma$
- 벨만 방정식 — 3장의 “점화식” 이 여기서 쓰입니다
- 가치 반복 / 정책 반복 (개념만)

교재

Sutton & Barto, *Reinforcement Learning: An Introduction* [무료 공개](#) — 1~4장이면 충분합니다

교수님께

1. 상태공간이  $2^{120}$  이라 정면으로 풀 수 없습니다. 기회비용을 근사하는 **가장 단순한** 방법이 무엇입니까? 학부 수준에서 방어 가능한 근사를 찾고 있습니다.

## 5 우리 주차장의 수학적 표현

추상적인 이야기를 실제 숫자로 내려놓습니다. 이 장이 교수님과의 대화에서 가장 유용할 것입니다.

### 5.1 그래프로서의 주차장

$$G = (V, E) \quad w(u, v) = \|\text{pos}(v) - \text{pos}(u)\|$$

방향 가중 그래프. 간선의 무게는 두 점 사이 유클리드 거리 — 벡터의 크기입니다.

| 기호           | 값     | 뜻                                              |
|--------------|-------|------------------------------------------------|
| $ V $        | 71    | 통로 위의 점. 여기에 통로 검지기가 묻혀 있다고 가정합니다              |
| $ E $        | 80    | 갈 수 있는 방향. <b>일방통행</b> 은 반대 간선이 <b>아예 없습니다</b> |
| 밀도           | 1.6 % | $80/71^2$ — 아주 희소한 그래프입니다                      |
| $S$          | 120   | 주차면 집합                                         |
| $a(s) \in V$ | —     | 주차면 $s$ 의 접근 노드. 그 자리 앞의 통로 위 점입니다             |
| $g$          | —     | 건물 출입구 좌표. 여기서 가까운 자리가 인기 자리입니다                |

구조는  $3 \times 3$  격자입니다. 수평 통로 3개는 **일방통행이 엇갈리게** 배치되어 있고(동 → 서 → 동), 수직 통로 5개가 그것을 잇습니다. 통로 안에서 U턴이 불가능하도록 일부러 이렇게 만들었습니다 — 그래야 “자리를 빼앗겨 되돌아가는 비용”이 실제로 발생하기 때문입니다.

### 5.2 문제의 정식화

주어진 것

$$I = \{1, \dots, n\} \quad J \subseteq S \quad v_i \in V$$

차량 집합 · 빈 자리 집합 · 각 차량이 지금 있는 통로 위의 점

## 비용

$$c(i, j) = w_{\text{drive}} \cdot d_G(v_i, a(j)) + w_{\text{walk}} \cdot \|\text{pos}(j) - g\|$$

앞의 항은 그래프 위 최단거리(타고 가는 거리), 뒤의 항은 직선 도보거리(내려서 걷는 거리)입니다.

## 찾는 것

$$\begin{aligned} \min_x \quad & \sum_{i \in I} \sum_{j \in J} c(i, j) x_{ij} \\ \text{s.t.} \quad & \sum_{j \in J} x_{ij} = 1 \quad (\forall i \in I) \\ & \sum_{i \in I} x_{ij} \leq 1 \quad (\forall j \in J) \\ & x_{ij} \in \{0, 1\} \end{aligned}$$

차량마다 자리를 정확히 하나씩, 한 자리에는 한 대만, 그러면서 총비용은 최소로.

### 여기서 이미 보이는 긴장

두 항이 서로 다른 자리를 가리킵니다. 실제 우리 도면에서 주행거리만 보면 F-01 이 최선이지만 거기서 건물까지 도보 103 m 입니다. 비용 전체로는 A-10 이 최선입니다(주행 131 m + 도보 12 m). 가중치  $w_{\text{walk}}$  를 얼마로 둘 것인가는 공학적 판단이지 수학이 답해 주지 않습니다. 너무 크면 모두 A구역으로 몰려 통로가 막히고, 너무 작으면 운전자가 안내를 무시합니다.

## 5.3 실행해 보기

동봉한 `parking_topology.py` 를 그대로 실행한 결과입니다. 차량 8대를 통로 곳곳에 흩어 놓고 인기 자리 20개를 두고 배정했습니다.

```
$ python parking_topology.py
```

#### == 4. 배정 : 탐욕법 vs 헝가리안 ==

차량 8 대가 통로 곳곳에 흩어져 있고, 빈 자리는 20 개

탐욕법    총비용 =        974.3

헝가리안 총비용 =        965.7    (-0.88 %)

배정이 어떻게 같았나:

| 차량 위치 | 탐욕법  | 헝가리안 |       |
|-------|------|------|-------|
| ENTRY | A-10 | A-10 |       |
| H2-04 | A-11 | A-11 |       |
| H2-11 | A-12 | A-12 |       |
| H2-18 | A-13 | A-14 | <- 다름 |
| H1-16 | A-14 | B-11 | <- 다름 |
| H1-08 | A-09 | A-08 | <- 다름 |
| H0-03 | A-08 | A-09 | <- 다름 |
| H0-14 | A-15 | A-13 | <- 다름 |

#### == 5. 혼잡 : 배정이 통로 부하를 바꾼다 ==

탐욕법    사용 간선 51 개, 최대 부하 4 대, 평균 1.61 대

헝가리안    사용 간선 50 개, 최대 부하 4 대, 평균 1.56 대

여덟 대 중 다섯 대의 배정이 같았고, 총비용은 0.88 % 개선됐습니다. 작아 보이지만 이건 *한 번의 배정*이고, 실제로는 900초 동안 180번 일어납니다. 그리고 더 중요한 것은 마지막 줄입니다 — **총비용이 거의 같아도 통로 부하 분포는 다릅니다**. 지금 비용 함수는 그 차이를 보지 못합니다.

#### 직접 해 볼 것

- 모든 차량을 ENTRY 에 두면 어떻게 됩니까? (힌트: 행이 전부 같아져 퇴화합니다. 어떤 배정이든 총비용이 같습니다. 왜 그런지 설명할 수 있으면 배정 문제를 이해한 것입니다.)
- w\_WALK 를 0 으로, 또 5.0 으로 바꿔 보세요. 배정이 어느 구역으로 몰립니까?
- 빈 자리를 20개가 아니라 8개로 줄이면 탐욕법과 헝가리안의 차이가 커집니까, 작아집니까? 왜 그럴까요?

## 6 교수님께 가져갈 질문지

모르는 것을 묻는 게 아니라, 어디까지 알아봤고 어디서 막혔는지를 보여주는 게 좋은 질문입니다.

### 먼저 보여드릴 것 (3분)

1. 문제 한 문장 — “빔 프로젝터로 바닥에 차량별 색 유도선을 그려 빈자리로 안내합니다. 인력 절감뿐 아니라 혼잡도 개선되는지 보고 싶습니다.”
2. 도면 — 노드 71 · 간선 80 · 주차면 120, 일방통행 격자
3. 지금 푸는 방식 — 가산적 비용 함수 + 탐욕적 배정
4. 측정한 결과 — 무안내 대비 주차소요 -47 %, 우회거리 -96 %, 처리량 +51 %
5. 막힌 지점 — “이게 혼잡 개선인지 확신이 없습니다”

### 질문 (우선순위 순)

| # | 분야   | 질문                                                                     |
|---|------|------------------------------------------------------------------------|
| 1 | 교통류  | 주차장의 “혼잡”을 정의하려면 무엇을 재야 합니까? 도로망의 기본도가 저속·빈번한 정차가 있는 주차장 통로에도 성립합니까?   |
| 2 | 큐잉   | 주차장을 $M/M/c/K$ 로 근사할 때 $c$ , $K$ 를 무엇으로 두어야 하며, 이 근사가 깨지는 지점은 어디입니까?   |
| 3 | 최적화  | 혼잡 항을 비선형(BPR 등)으로 바꾸면 배정 문제의 정수성이 깨집니다. 그래도 실용적으로 풀 수 있는 방법이 있습니까?    |
| 4 | 균형   | 저희는 사회적 최적을 강제할 수 있는 경우라고 봅니다. 무정부의 대가를 실제로 측정하려면 분모(사후 최적)를 어떻게 구합니까? |
| 5 | 온라인  | 도착 분포를 아는 온라인 배정에서, 최악 경우가 아니라 기댓값 기준의 성능 보장을 주는 틀이 있습니까?              |
| 6 | MDP  | 자리의 기회비용을 학부 수준에서 방어 가능하게 근사하는 방법은?                                    |
| 7 | 실험설계 | “혼잡이 줄었다”를 주장하려면 어떤 실험이 필요합니까? 도착률을 올려가며 포화점을 비교하려는데 타당한 접근입니까?        |
| 8 | 검증   | 시뮬레이션 결과를 논문·공모전에서 방어하려면 어떤 검증(validation)이 필요합니까?                     |

#### 질문할 때

- “이거 어떻게 해요?” 보다 “**저희는 이렇게 생각했는데 맞습니까?**” 가 훨씬 좋은 답을 얻습니다. 틀려도 괜찮습니다 — 어디가 틀렸는지 알려줍니다.
- 답을 다 이해 못 해도 **키워드를 적어 오세요.** “그건 ○○ 문제네요” 라는 한마디가 한 달치 검색을 줄여 줍니다.
- `parking_topology.py` 를 노트북에 띄워 가세요. 화면에 숫자가 있으면 대화가 구체적이 됩니다.



# 7 4주 학습 계획과 자료

하루 1~2시간 기준입니다. 완벽히 이해하려 하지 말고 일단 끝까지 통과하세요.

| 주  | 주제        | 할 일                                                                                                |
|----|-----------|----------------------------------------------------------------------------------------------------|
| 1주 | 표기법 + 그래프 | 3장의 표기 익히기 · 그래프 표현과 다익스트라 · <code>parking_topology.py</code><br>1~2절을 직접 고쳐 보기 (다른 출발점에서 최단경로 뽑기) |
| 2주 | 배정 문제     | 이분 매칭 · 헝가리안(원리보다 사용법 먼저) · LP 와 쌍대성 맛보기 · 코드<br>4절을 바꿔 가며 실험                                      |
| 3주 | 확률과 큐잉    | 포아송 과정 · 리틀의 법칙 · M/M/c · 우리 실험 결과에 직접 대입해 보기                                                      |
| 4주 | 교통류와 균형   | 기본도 · BPR · Wardrop UE/SO · Sheffi 1~2장 · 질문지 정리해서<br>교수님 면담                                       |

## 자료 목록

| 자료                                                           | 분야       | 비고                              |
|--------------------------------------------------------------|----------|---------------------------------|
| Sheffi, <i>Urban Transportation Networks</i>                 | 교통류 · 균형 | 무료 공개 4.5의 표준서. 우리 문제와 가장 가깝습니다 |
| Sutton & Barto, <i>Reinforcement Learning</i>                | MDP      | 무료 공개 1~4장만                     |
| Kleinberg & Tardos, <i>Algorithm Design</i>                  | 그래프 · 매칭 | 가장 읽기 쉽습니다. 4장(그래프), 7장(흐름·매칭)  |
| CLRS, <i>Introduction to Algorithms</i>                      | 알고리즘 전반  | 통독용이 아니라 참고서. 22~26장            |
| Papadimitriou & Steiglitz, <i>Combinatorial Optimization</i> | 조합 최적화   | Dover 보급판이 저렴합니다                |
| Ahuja · Magnanti · Orlin, <i>Network Flows</i>               | 흐름 · 배정  | 최소비용 흐름을 제대로 보려면                |
| Ross, <i>Introduction to Probability Models</i>              | 확률 · 큐잉  | 포아송과 대기행렬                       |
| Borodin & El-Yaniv, <i>Online Computation</i>                | 온라인      | 필요할 때만                          |

한 가지만 고른다면

**Sheffi** 의 *Urban Transportation Networks* 입니다. 무료로 공개돼 있고, “여러 사람이 각자 최선을 택할 때 전체가 어떻게 되는가”라는 우리 문제의 핵심을 정면으로 다룹니다. 앞의 두세 장만 읽어도 무안내와 유도선 안내의 차이를 설명할 언어가 생깁니다.

---

이 문서와 `parking_topology.py` 는 프로젝트 저장소의 `docs/study/` 에 있습니다.